



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO · FACULTAD DE CIENCIAS

COMPILADORES

PRÁCTICA 1 · 2027-1

Infraestructura básica del analizador léxico de MiniC

INTEGRANTES

Jesus Antonio Romero Godoy	321144292
Gael Emiliano Arreguin Salgado	321121721

PROFESORA: L. en C.C. Yessica Janeth Pablo Martínez

AYUDANTE DE LABORATORIO: Pedro Yamil Salazar González

FECHA DE ENTREGA: 5 de Septiembre del 2026

1. OBJETIVO

Esta práctica tiene como objetivo diseñar e implementar la infraestructura básica del analizador léxico (*lexer*) del lenguaje MiniC, el lenguaje que se usará durante todo el curso para construir, de forma incremental, las distintas etapas de un compilador. En esta primera entrega se partió desde la representación de tokens, la lectura de archivos fuente, el seguimiento de la posición (línea y columna) dentro del archivo, y se construyó el reconocimiento de un subconjunto inicial de la gramática léxica de MiniC: espacios en blanco, símbolos simples de un carácter, números enteros sin signo, caracteres no reconocidos (ERROR) y el final del archivo (TOKEN_EOF).

2. DESCRIPCIÓN DE LA SOLUCIÓN

2.1 Organización de módulos

El proyecto se organizó siguiendo la estructura mínima solicitada en la especificación, separando claramente tres responsabilidades:

- `include/lexer/token.h`, `src/lexer/token.c`: representación de tokens (*qué es un token y cómo se crea, imprime y libera*).
- `include/lexer/lexer.h`, `src/lexer/lexer.c`: el analizador léxico propiamente dicho (*cómo se recorre el archivo fuente y se reconocen los tokens*).
- `src/main.c`: punto de entrada. Se limita a validar los argumentos de línea de comandos, abrir el archivo fuente, invocar al analizador léxico y liberar los recursos utilizados — toda la lógica de análisis vive en los módulos del `lexer`, nunca en `main.c`.

2.2 Representación de tokens

Cada token se representa mediante la estructura:

```
1 typedef struct {  
2     TokenType type;  
3     char *lexeme;  
4     size_t line;  
5     size_t column;  
6 } Token;
```

`TokenType` es una enumeración con las 29 categorías léxicas exigidas por la especificación. Aunque todas están definidas desde esta entrega (no se modificó la enumeración, conforme a las restricciones de la práctica), en esta primera versión solo se reconocen efectivamente: `INTEGER`, los operadores `PLUS`, `MINUS`, `STAR`, `SLASH`, `ASSIGN`, los relacionales `LESS` y `GREATER`, los delimitadores `LPAREN`, `RPAREN`, `LBRACE`, `RBRACE` y `SEMICOLON`, además de `ERROR` y `TOKEN_EOF`.

Tres funciones administran el ciclo de vida de un token:

- `token_init`: reserva memoria para una copia propia del lexema (con `malloc` y `memcpy`) y valida que la reserva haya tenido éxito.
- `token_print`: imprime el token en el formato exacto exigido por la especificación (`línea:columna TIPO lexema`), con el caso especial de `TOKEN_EOF`, que no tiene lexema.
- `token_destroy`: libera la memoria del lexema una vez que el token ya no se necesita.

2.3 Estrategia para recorrer el archivo

El archivo se recorre con un único ciclo que consume caracteres de uno en uno mediante `fgetc`. Dos casos requieren decidir qué hacer con un carácter *después* de haber visto el siguiente, sin haber decidido aún si ese carácter adicional pertenece al token actual:

- Distinguir un `\r` aislado de la secuencia `\r\n`.
- Saber dónde termina una secuencia de dígitos consecutivos (`[0-9]+`).

Ambos casos se resuelven con el mismo patrón: leer el siguiente carácter con `fgetc` y, si no pertenece al token o evento actual, regresarlo al flujo con `ungetc` para que la siguiente vuelta del ciclo principal lo procese con normalidad. Este patrón de “espiar y regresar” se mantuvo centralizado en el ciclo principal y en la función auxiliar `scan_integer`, dejando otras funciones (como `advance_position`) libres de manejar entrada/salida directamente.

2.4 Seguimiento de línea y columna

Las líneas se numeran a partir de 1 y las columnas a partir de 0, de acuerdo con la especificación. *Todo* carácter leído del archivo — incluidos los espacios en blanco — actualiza la posición antes de decidir si genera un token o se ignora, por lo que la posición siempre refleja con exactitud dónde se encuentra el analizador.

El caso más delicado es el salto de línea: `\n` y un `\r` aislado deben incrementar la línea y reiniciar la columna a 0, pero la secuencia `\r\n` debe contarse como un *único* salto de línea, no como dos. Nuestra solución fue espiar el carácter siguiente en cuanto se lee un `\r`: si es `\n`, se consume ahí mismo (nunca vuelve a aparecer como carácter independiente en el ciclo), si no lo es, se regresa con `ungetc`. Con eso, la función que actualiza línea y columna solo necesita tratar `\n` y `\r` como equivalentes, y no tiene que preocuparse de si vinieron combinados:

Gracias a que *todo* carácter (incluidos los espacios y saltos finales) pasa por `advance_position`, la posición de `TOKEN_EOF` “la posición inmediatamente posterior al último carácter procesado, después de ignorar cualquier espacio en blanco final” se obtiene sin ningún cálculo adicional: es, simplemente, el valor de `line/column` en el momento en que `fgetc` regresa EOF.

2.5 Manejo de lexemas y memoria

Los símbolos de un carácter (+, ;, etc.) y el token `ERROR` usan un lexema de un solo carácter, construido en un buffer de pila de tamaño fijo (`char lexeme[2]`). Los enteros, en cambio, pueden tener cualquier cantidad de dígitos, por lo que `scan_integer` construye el lexema en un *buffer* que crece dinámicamente:

La capacidad se duplica cuando hace falta, siempre dejando espacio para el carácter terminador `'\0'`. Como `token_init` copia el contenido del lexema hacia memoria propia del Token, el *buffer* temporal devuelto por `scan_integer` se libera inmediatamente después de esa copia, sin importar si la copia tuvo éxito o no evitando tanto fugas de memoria como liberaciones dobles. Cada Token impreso se libera con `token_destroy` antes de procesar el siguiente carácter.

2.6 Tratamiento de errores y final del archivo

Cualquier carácter que no sea espacio en blanco, símbolo simple ni dígito genera un token `ERROR` que conserva ese carácter como lexema junto con su posición exacta, y el análisis continúa normalmente con el siguiente carácter, es decir, no se interrumpe la ejecución ni se mezclan estos diagnósticos con mensajes en `stderr`. Al llegar al final del archivo (`fgetc` devuelve EOF), se emite

un único token `TOKEN_EOF` con la posición descrita en la sección anterior, y solo si no ocurrió un error de lectura (`ferror`).

3. RESULTADOS Y PRUEBAS

El proyecto incluye dos conjuntos de pruebas automatizadas, ejecutables con `make test`, ambos con la misma convención (`inputs/*.mc + expected/*.out`):

- `tests/public/`: 10 casos mínimos proporcionados por el curso.
- `tests/propias/`: 13 casos escritos por nosotros, uno por cada punto mínimo exigido en la sección 12 de la especificación (archivo vacío, solo espacios en blanco, un único token, todos los símbolos simples, entero de un dígito, entero de varios dígitos, varios tokens en una línea, tokens en varias líneas, combinación de espacios y tabuladores, un carácter no reconocido, varios caracteres no reconocidos, posición correcta de línea/columna, y detección correcta del final del archivo), diseñados para no depender únicamente de los ejemplos ya proporcionados.

Al ejecutar `make clean && make && make test` las 23 pruebas pasan.

Como ejemplo concreto del formato de salida, la prueba `t10_error_uno.mc` contiene `5 # 3;` (un carácter no reconocido en medio de una expresión válida) y produce:

```
1 $ ./minic tests/propias/inputs/t10_error_uno.mc
2 1:0 INTEGER 5
3 1:2 ERROR #
4 1:4 INTEGER 3
5 1:5 SEMICOLON ;
6 1:6 TOKEN_EOF
```

Este ejemplo muestra, en un solo caso, tres de los requisitos más importantes de la práctica: el reconocimiento de enteros, la generación de `ERROR` sin detener el análisis, y el cálculo correcto de columna incluso después de un carácter no reconocido.

4. PROBLEMAS ENCONTRADOS

- **Doble conteo de `\r\n`.** El reto principal del seguimiento de posición no fue reconocer un `\r` aislado (bastaba con tratarlo igual que `\n`), sino evitar contar dos veces la secuencia `\r\n`, ya que ambos caracteres se leen por separado con `fgetc`. Lo resolvimos espiando el carácter siguiente, en cuanto se detecta un `\r` y, de ser `\n`, se consume de inmediato para que nunca llegue a procesarse como un salto de línea independiente.
- **Lexemas de longitud variable.** A diferencia de los símbolos de un carácter, un entero puede tener cualquier cantidad de dígitos, lo que un buffer de tamaño fijo no puede garantizar de forma segura. Optamos por un buffer que crece dinámicamente (`malloc/realloc` con duplicación de capacidad) en vez de fijar un límite arbitrario de dígitos.
- **Convención de archivos de prueba.** La regla `test` del `Makefile`, tal como venía en la base inicial, buscaba pares `archivo.mc/archivo.expected` en un único directorio plano, lo cual no coincidía con la estructura de las pruebas públicas proporcionadas (`tests/public/inputs/`

+ tests/public/expected/*.out). Generalizamos la regla a un bucle parametrizado por directorio que recorre tanto tests/propias/ como tests/public/ bajo el mismo esquema inputs//expected/.

5. CONCLUSIONES

Logramos construir una primera versión funcional y completa del analizador léxico de MiniC dentro del alcance definido para esta práctica, validada con 23 pruebas automatizadas (10 públicas y 13 propias), todas exitosas. La organización modular adoptada: separación entre el módulo de tokens, el módulo del lexer y main.c deja una base clara para la siguiente practica.

6. USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

Herramienta ChatGPT

Propósito La utilizamos principalmente para entender de manera más completa el código base con el que partíamos, y así tener un contexto general de toda la práctica y comprender bien su estructura. Además, como no contábamos con experiencia previa programando en C, también sirvió para ver cómo se organiza y estructura un proyecto en este lenguaje (separación en módulos, uso de archivos .h/.c, compilación con Makefile, etc.), lo cual guio la forma en que se implementó y extendió el código.

7. REFERENCIAS

1. Salazar González, P. Y. *Compiladores Practica1*. Compiladores, 2027-1, Facultad de Ciencias, UNAM.
2. ISO/IEC 9899:2011 (C11). Secciones 7.4 (ctype.h) y 7.21 (stdio.h): especificación de isdigit, fgetc, ungetc, ferror.